

urdu-slm

A from-scratch Urdu language model

Brief — [concise technical overview](#)

Project by Salman Adnan

Document generated 2026-07-07

urdu-slm builds a small Urdu-language transformer entirely from scratch: a data pipeline that cleans and deduplicates public Urdu text, a 32k-vocabulary byte-level BPE tokenizer trained on that corpus, a decoder-only transformer implemented directly in PyTorch (no borrowed modeling code), a resumable training loop, and an evaluation harness. It exists to demonstrate the parts of “train a GPT” that English-first tutorials usually skip: script normalization, a tokenizer that does not fragment on non-Latin script, and an honest reckoning with a small training corpus.

1 Pipeline at a glance



The corpus (real numbers). From Urdu Wikipedia (1.73M articles) and two Leipzig newscrawl corpora (1.3M sentences combined), extraction and filtering (NFC normalization, a 65%-Arabic-script-ratio language filter, boilerplate removal) plus exact + MinHash near-deduplication removed 31.9% of extracted paragraphs, leaving **2.49M train / 25.1k val documents**, 438.5M characters, tokenizing to **111.7M training tokens**.

The tokenizer. A 32k byte-level BPE trained on the cleaned corpus achieves 4.078 chars/token on Urdu, versus 0.779 for GPT-2’s tokenizer on the same text — GPT-2 needs **5.23x** more tokens.

2 The model

A pre-norm decoder-only transformer: RMSNorm, RoPE (rotary position embeddings) causal self-attention via `scaled_dot_product_attention`, SwiGLU MLP (8/3 hidden ratio), tied input/output embeddings.

Key terms, briefly

RMSNorm rescales activations to a stable range each layer. **RoPE** gives attention a notion of token position by rotating query/key vectors. **SwiGLU** is a gated feed-forward layer, an alternative to GPT-2’s GELU MLP. **Tied embeddings** share one weight matrix between the input token lookup and the output prediction layer, halving the vocabulary’s parameter cost.

Preset	Params	Layers	d_{model}	Heads	Context
tiny	13.0M	6	256	4	512
small	35.7M	6	512	8	1024
base	123.7M	14	768	12	1024

Honest read

At a real 32k vocabulary, the tied embedding table is $\sim 8.2\text{M}$ parameters on its own — so **tiny**’s 13.0M total is dominated by vocabulary, not transformer depth. This is reported directly rather than hidden by shrinking the vocabulary to hit a rounder number.

3 Training

train.py schedules training by **token budget**, not raw step count; supports bf16 autocast on GPU (fp32 on CPU), gradient accumulation, cosine LR with warmup, gradient clipping, full resume (model + optimizer + data-loader cursor), and multi-GPU DDP via **torchrun**. Every config in **configs/** sets a token budget; the model, learning-rate schedule, and JSONL metrics logging are otherwise identical across presets.

4 Runs actually executed

4.1 Phase 1 — tiny (13M), CPU: done

500 steps / 4.10M tokens on this machine’s CPU. Train loss $9.74 \rightarrow 5.42$; held-out perplexity 248.8; 98.2% of generated text in Arabic script; cloze accuracy 0.378 (chance 0.333). Purpose: prove the whole pipeline works end-to-end, not to produce a strong model — and it does.

4.2 Phase 2 — base (123.7M), GPU: completed

Status

Earlier project documentation described the phase-2 GPU run as “not run yet.” That is now out of date: **the run has been completed and its results are committed to the repository.**

Config **configs/base.yaml** targets 450M tokens (≈ 4 epochs of the 111.7M-token corpus, a deliberately repeat-bounded budget — see below), 858 total steps at an effective 524,288 tokens/step. The run executed on a rented RunPod A100 SXM 80GB GPU on 2026-07-07: all 858 steps completed in about 75 minutes at a sustained $\approx 99,470$ tokens/second, for a real cost of about \$1.90.

Committed log: **runs/base/metrics.jsonl** logs every 20 steps through step 840 of 858 (the last 18 steps to completion were not on a logging boundary and are not in the machine-readable file). At step 840: 440.4M of the 450M-token budget, loss **3.5087**, falling monotonically the whole way from 9.59. No **val_loss** entries appear in this file — the configured **eval_interval** (1000) was never reached within 858 total steps, so held-out evaluation was *not* logged automatically during training; it was run separately afterward (below). Checkpoint files (**runs/base/ckpt.pt**, 1.48GB; **runs/base/model.pt**, 495MB) are timestamped after the last logged metric and are the expected size for a completed 123.7M-parameter checkpoint.

Why 450M tokens, not more: the Chinchilla heuristic suggests ~ 20 tokens/parameter ($\sim 2.5\text{B}$ for this model); the corpus has only 0.9 tokens/parameter. Reaching Chinchilla scale would mean repeating



the corpus ~ 22 times, which risks overfitting; `docs/GPU_RUN.md` instead targets ~ 4 epochs (repetition research suggests that repeats up to about 4 epochs costs little), an explicitly **data-limited**, **not compute-limited** budget. More corpus, not more GPU time, is the documented next lever.

Post-training evaluation (run separately with this project’s own `eval/` scripts against the trained checkpoint):

Metric	Result
Held-out perplexity (<code>data/val.bin</code>)	38.02
Script mix (20 generations)	99.43% Arabic script
Cloze accuracy (2-shot, 43 items)	34.9% (chance $\approx 33.3\%$)

Honest read

Read plainly: the model reliably writes in Urdu’s actual script and has learned real Urdu grammar and vocabulary, but perplexity and cloze accuracy are modest — expected for a 124M model trained on a data-limited corpus for one ~ 4 -epoch run, not evidence of strong language understanding. The project’s own sample generations (in the README) show fluent local grammar but topical drift from the prompt, and at least one visible repetition-loop failure mode. This is reported as the honest result, not smoothed over.

5 Testing and licensing

12/12 tests pass (5 model: shapes, causal masking, tied embeddings, generation, parameter-count bands; 7 pipeline: normalization, script-ratio filtering, exact + near dedup). Code is MIT-licensed; Urdu Wikipedia data is CC BY-SA 4.0 and the Leipzig corpora are CC BY-NC 4.0, so released weights inherit a non-commercial restriction from the Leipzig portion (see `docs/DATA_LICENSES.md`).

6 Bottom line

Status

Phase 1 (pipeline, tokenizer, CPU proof run): done and verified in this repository. **Phase 2** (the 124M **base** model): trained on a real GPU, 858 steps / 449.8M tokens, loss falling cleanly to 3.51 (last logged step 3.5087 at step 840), evaluated with modest but honest held-out numbers (perplexity 38.02, cloze 34.9%, 99.4% correct script). The clearest documented next step is growing the corpus, not spending more compute on the current one.